

TRABAJO DE TITULACIÓN — UNIDAD 3

Shunayka G. Baquero

Tema: Sistema Web de Control de Inventario

Inteligente con Escáner de Cámara, Predicción de Stock y Comandos de Voz Accesible
para Tiendas de Abastos Locales

Subtemas: Caso práctico para aplicación de escritorio, web y móvil.

Instituto Superior Universitario Japón

Ing. Luis Calo

Septiembre 19, 2026



Índice de Contenido

Índice de Contenido.....	2
CONTEXTUALIZACIÓN DEL CASO.....	5
OBJETIVO DEL CASO.....	5
Objetivo general:.....	5
Objetivos específicos:.....	6
ESCENARIO O SITUACIÓN.....	6
Situación actual:.....	6
Problemáticas identificadas:.....	6
Propuesta de solución:.....	7
PREGUNTAS GUÍA O RETOS.....	7
1. ¿Cuáles son los requerimientos funcionales y no funcionales esenciales para garantizar que la aplicación cumpla con las necesidades del usuario final y los objetivos del proyecto?.....	7
Funcionales esenciales:.....	8
No funcionales esenciales:.....	8
2. ¿Qué metodología de desarrollo de software es la más adecuada para este proyecto y cómo influye en la planificación y organización del trabajo?.....	8
3. ¿Cómo estructurar y diseñar correctamente la base de datos para asegurar integridad, eficiencia en consultas y escalabilidad del sistema?.....	9
4. ¿Qué arquitectura, paradigma de programación y patrón de diseño permiten construir una aplicación escalable, mantenible y segura?.....	10
5. ¿Cómo debe diseñarse la interfaz de usuario para asegurar una experiencia fluida, intuitiva y accesible para el usuario final?.....	10
6. ¿Qué estrategias deben seguirse para una correcta implementación, documentación y presentación del sistema ante los usuarios o evaluadores?.....	11
1. ANÁLISIS Y DISEÑO.....	12
1.1 Estudio de factibilidad.....	12
1.1.1 Factibilidad técnica.....	12
1.1.2 Factibilidad económica.....	12
1.1.3 Factibilidad operativa.....	13
1.1.4 Factibilidad legal.....	13
1.2 Requerimientos funcionales y no funcionales.....	13
1.2.1 Requerimientos funcionales.....	13
Tabla 1.....	13
1.2.2 Requerimientos no funcionales.....	14
Tabla 2.....	14
1.3 Metodología de desarrollo seleccionada.....	15
Tabla 3.....	15





1.4 Diagramas UML.....	16
1.4.1 Diagrama de casos de uso.....	16
Figura 1.....	16
1.4.2 Diagrama de clases.....	17
Figura 2.....	17
2. DISEÑO DE BASE DE DATOS.....	19
2.1 Modelo Entidad-Relación (ME-R).....	19
Figura 3.....	19
2.2 Modelo Relacional.....	21
Tabla 4.....	21
Tabla 5.....	21
Tabla 6.....	22
Tabla 7.....	22
Tabla 8.....	23
Tabla 9.....	24
Tabla 10.....	24
Tabla 11.....	25
2.3 Script SQL de creación.....	26
3. ARQUITECTURA Y DISEÑO TÉCNICO.....	31
3.1 Tipo de arquitectura del sistema.....	31
Figura 4.....	31
3.1.1 Núcleo de dominio.....	32
3.1.2 Puertos.....	32
3.1.3 Adaptadores primarios (driving).....	33
3.1.4 Adaptadores secundarios (driven).....	33
3.2 Paradigma de programación.....	34
3.2.1 Programación Orientada a Objetos (POO).....	34
3.2.2 Programación Funcional.....	34
3.3 Patrones de diseño aplicados.....	35
Tabla 12.....	35
3.4 Framework de desarrollo.....	35
3.4.1 Backend.....	35
Figura 5.....	35
3.4.2 Frontend.....	37
Figura 6.....	37
3.5 Escalabilidad multiplataforma.....	40
Tabla 13.....	40
4. DISEÑO DE INTERFAZ.....	40



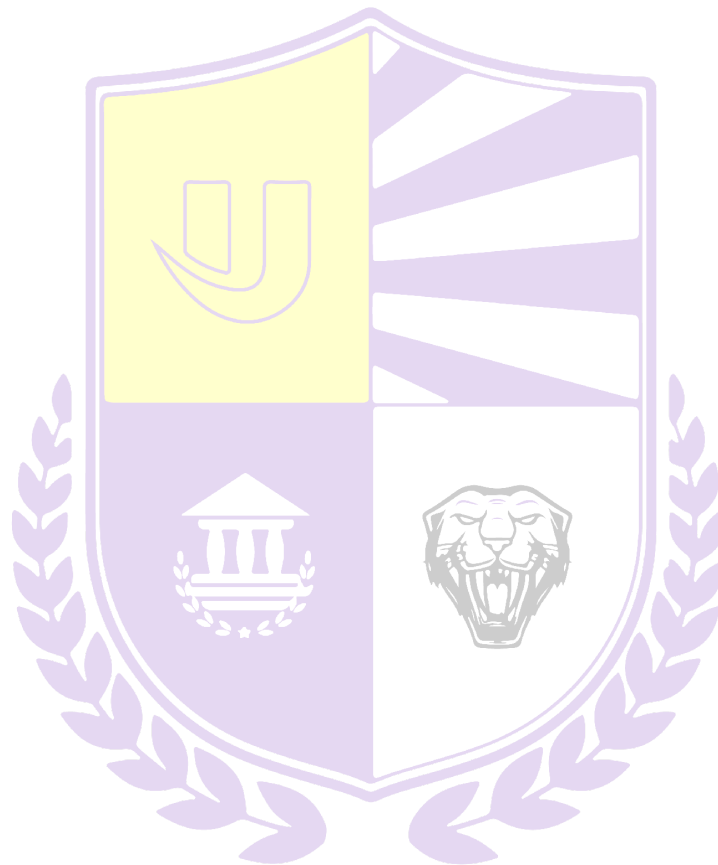


4.1 Mockups de usuario.....	40
4.1.1 Pantalla de inicio de sesión.....	41
Figura 7.....	41
4.1.2 Pantalla de registro.....	42
Figura 8.....	42
4.1.3 Panel principal (Dashboard).....	43
Figura 9.....	43
4.1.4 Módulo de inventario.....	44
Figura 10.....	44
4.1.5 Módulo de comandos de voz.....	44
Figura 11.....	44
4.2 Prototipo de Front End funcional.....	45
4.2.1 Funcionalidades implementadas.....	46
Tabla 14.....	46
4.2.2 Accesibilidad aplicada.....	46
4.2.3 Cómo acceder al prototipo.....	47
5. IMPLEMENTACIÓN.....	50
5.1 Proceso de implementación (escritorio, web y móvil).....	50
5.1.1 Implementación web.....	50
5.1.2 Implementación de escritorio.....	52
5.1.3 Implementación móvil.....	53
5.2 Conexión a base de datos.....	54
5.2.1 Configuración de la conexión.....	54
5.2.2 Servicio de conexión en NestJS.....	55
5.2.3 Flujo completo de una operación.....	56
5.2.4 Migraciones y datos semilla.....	57
5.2.5 Despliegue en producción.....	57
6. DOCUMENTACIÓN Y PRESENTACIÓN.....	58
6.1 Estrategia de documentación.....	58
6.1.1 Documentación técnica.....	58
6.1.2 Documentación de la API.....	58
6.1.3 Manual de usuario.....	59
Tabla 15.....	59
6.2.2 Recomendaciones para la defensa.....	60
7. CONCLUSIONES Y RECOMENDACIONES.....	60
7.1 Conclusiones.....	60
7.2 Recomendaciones.....	61
8. BIBLIOGRAFÍA.....	62





9. ANEXOS.....	63
Tabla 16.....	63
Tabla 17.....	64





CONTEXTUALIZACIÓN DEL CASO

El presente trabajo aborda el desarrollo de un sistema web de control de inventario inteligente orientado a tiendas de abastos locales del Ecuador, un sector económico tradicionalmente desatendido por las soluciones tecnológicas actuales. Estas tiendas representan aproximadamente el 60 % del comercio minorista del país y, sin embargo, la mayoría sigue operando con métodos manuales de registro de mercancías, lo que genera pérdidas por desabastecimiento, vencimiento de productos y descontrol del stock.

La problemática se agrava porque los tenderos atienden simultáneamente el mostrador, la caja y el repositorio de productos, lo que les impide dedicar tiempo a tareas administrativas complejas. Los sistemas ERP comerciales existentes están diseñados para cadenas de supermercados, con interfaces saturadas, licencias costosas y requisitos de hardware que resultan inalcanzables para el pequeño comerciante.

Antecedente: La pandemia de COVID-19 evidenció la importancia del comercio de proximidad, pero también la brecha digital de las tiendas de barrio. Un sistema accesible, económico y diseñado específicamente para el contexto del tendero ecuatoriano puede convertirse en un aporte significativo a la digitalización del comercio local.

OBJETIVO DEL CASO

Objetivo general:

Diseñar y desarrollar un sistema web de control de inventario inteligente para tiendas de abastos locales, que integre un escáner de códigos de barras mediante cámara, un módulo predictivo de desabastecimiento y un control por comandos de voz, cumpliendo con las





normativas de accesibilidad WCAG 2.1 nivel AA y los principios heurísticos de usabilidad de Nielsen.

Objetivos específicos:

Automatizar el registro de mercancías mediante lectura de códigos de barras por cámara y comandos de voz nativos, reduciendo el tiempo de ingreso manual en al menos 60 %.

Diseñar un algoritmo predictivo que evalúe los patrones de venta históricos y emita alertas tempranas de productos próximos a agotarse.

Garantizar una experiencia de usuario inclusiva cumpliendo los criterios WCAG 2.1 AA y los diez principios heurísticos de Nielsen.

Desarrollar una arquitectura escalable basada en el patrón hexagonal que permita desplegar el sistema en escritorio, web y móvil con un mismo núcleo de negocio.

ESCENARIO O SITUACIÓN

Situación actual:

Doña María regenta una tienda de abastos en el sur de Quito desde hace 15 años. Atiende a más de 80 clientes diarios en un local de 20 m². Cada vez que llega mercadería del distribuidor, debe verificar manualmente las facturas y anotar en un cuaderno las cantidades recibidas. En horas pico, cuando hay 3 o 4 personas esperando, no tiene tiempo de registrar las ventas con precisión. Al final del día, debe cuadrar el inventario "a ojo" y, cuando se da cuenta de que le falta un producto, ya es demasiado tarde: los clientes se han ido a la tienda de la esquina.

Problemáticas identificadas:





Desabastecimiento no planificado: productos que se agotan sin previo aviso, generando pérdida de ventas.

Vencimiento de mercancía: productos perecederos que caducan porque no se rota el stock por fecha.

Registro manual lento: doña María pierde entre 15 y 20 minutos diarios solo en registro de inventario.

Sin trazabilidad: no hay historial de ventas que permita tomar decisiones informadas.

Exclusión digital: las interfaces de los sistemas comerciales existentes son complejas, con tipografías pequeñas y saturación visual, lo que las hace inaccesibles para usuarios con visión reducida o poca familiaridad tecnológica.

Propuesta de solución:

Un sistema web progresivo (PWA) que aproveche la cámara del celular para escanear códigos de barras, la API de voz del navegador para operar manos libres, y un algoritmo simple de rotación para predecir el desabasto. Todo con una arquitectura desacoplada que permita su despliegue futuro como app nativa sin reescribir el núcleo de negocio.

PREGUNTAS GUÍA O RETOS

1. ¿Cuáles son los requerimientos funcionales y no funcionales esenciales para garantizar que la aplicación cumpla con las necesidades del usuario final y los objetivos del proyecto?

Los requerimientos funcionales (RF) y no funcionales (RNF) se detallan en la sección 1.2 del presente documento. De forma resumida:



**Funcionales esenciales:**

RF-01: Gestión de autenticación y roles (Administrador y Tendero).

RF-02: Registro de productos con escáner de código de barras.

RF-03: Procesamiento de comandos de voz.

RF-04: Registro de movimientos de entrada/salida.

RF-05: Alertas predictivas de desabastecimiento.

RF-06: Configuración de accesibilidad visual.

No funcionales esenciales:

RNF-01: Cumplimiento WCAG 2.1 nivel AA.

RNF-02: Diseño responsivo (escritorio, tablet, móvil).

RNF-03: Latencia del escáner < 2 segundos.

RNF-04: Integridad transaccional en la base de datos.

RNF-05: Contraseñas cifradas con hash.

RNF-06: Compatibilidad con Chromium y Firefox.

2. ¿Qué metodología de desarrollo de software es la más adecuada para este proyecto y cómo influye en la planificación y organización del trabajo?

Se seleccionó Scrum adaptado a un desarrollo individual. Si bien Scrum fue diseñado originalmente para equipos multidisciplinarios con tres roles diferenciados, su marco de trabajo





permite adaptaciones válidas en contextos académicos y de equipos pequeños, manteniendo intactos sus principios de iteración, inspección y adaptación.

En este proyecto, la autora asume simultáneamente los tres roles de Scrum: Product Owner (prioriza el backlog según las necesidades del tendero), Scrum Master (gestiona plazos y remueve impedimentos) y Developer (modela y construye la solución).

Esta metodología permite entregas modulares e incrementales cada dos semanas, facilita la incorporación temprana de hallazgos de usabilidad y accesibilidad, y se adapta a la naturaleza cambiante de los requerimientos del tendero. El detalle de los sprints, roles y la justificación completa se desarrolla en la **sección 1.3.**

3. ¿Cómo estructurar y diseñar correctamente la base de datos para asegurar integridad, eficiencia en consultas y escalabilidad del sistema?

La base de datos se diseñó en PostgreSQL siguiendo las siguientes decisiones:

Normalización hasta 3FN: se eliminaron dependencias transitivas y se separaron entidades como Rol y Categoría para evitar redundancia.

Integridad referencial: claves foráneas con ON DELETE RESTRICT (para roles y productos) y ON DELETE CASCADE (para configuraciones y alertas).

Índices: se crearon índices en columnas de búsqueda frecuente (correo, codigo_barras, id_producto en movimientos).

Restricciones CHECK: validan rangos (precio_venta $\geq$ 0, stock_actual $\geq$ 0, tipo IN ('ENTRADA','SALIDA','AJUSTE')).

Prefijo sgb_: todas las tablas llevan las iniciales de la autora, según exige el taller.





El modelo E-R y el script SQL completo se presentan en la sección 2.

4. ¿Qué arquitectura, paradigma de programación y patrón de diseño permiten construir una aplicación escalable, mantenible y segura?

Se aplica arquitectura hexagonal (puertos y adaptadores), que separa: Núcleo de dominio: entidades y reglas de negocio puras.

Puertos: interfaces que definen contratos (ej. IUserRepository).

Adaptadores primarios (driving): controladores REST, UI.

Adaptadores secundarios (driven): Prisma, bcrypt, Web Speech API.

Paradigma: combina POO (para modelado de entidades) y programación funcional (para lógica de negocio y hooks).

Patrones aplicados:

Repository: encapsula el acceso a datos.

Dependency Injection: NestJS inyecta dependencias entre capas.

Use Case: cada caso de uso es una clase con un método ejecutar().

Adapter: permite cambiar de ORM, framework o UI sin tocar el dominio.

El framework backend es NestJS y el frontend React + Vite + TypeScript (ver sección 3).

5. ¿Cómo debe diseñarse la interfaz de usuario para asegurar una experiencia fluida, intuitiva y accesible para el usuario final?

Se aplican los diez principios heurísticos de Nielsen (1994) y las WCAG 2.1 nivel AA:





Contraste mínimo de 4.5:1 entre texto y fondo.

Tipografía escalable hasta el 200 % sin perder funcionalidad.

Navegación completa por teclado con foco visible.

Etiquetas ARIA para lectores de pantalla.

Mensajes de error en lenguaje natural con sugerencias de solución.

Diseño minimalista: solo alertas críticas y resumen diario en el panel principal.

Se diseñaron mockups en Figma y un prototipo funcional en React (ver sección 4).

6. ¿Qué estrategias deben seguirse para una correcta implementación, documentación y presentación del sistema ante los usuarios o evaluadores?

Implementación:

Despliegue del backend en un VPS con Node.js + PostgreSQL.

Frontend como PWA (Progressive Web App) para funcionar en escritorio, web y móvil.

Uso de Docker para empaquetar la aplicación y sus dependencias.

CI/CD con GitHub Actions para pruebas automatizadas.

Documentación:

README.md con instrucciones de instalación y ejecución.

Documentación de la API con Swagger/OpenAPI.

Manual de usuario con capturas de pantalla.

Comentarios JSDoc en el código.





Presentación:

Exposición estructurada en 5 bloques: problema, solución, arquitectura, demo, conclusiones.

Preparación de respuestas a las preguntas más probables del tribunal.

Ensayo previo con cronómetro para cumplir el tiempo asignado.

1. ANÁLISIS Y DISEÑO

1.1 Estudio de factibilidad

El estudio de factibilidad evalúa la viabilidad del proyecto en cuatro dimensiones.

1.1.1 Factibilidad técnica

El proyecto se desarrolla con tecnologías open source y ampliamente documentadas: NestJS con Node.js 20 LTS para el backend, PostgreSQL 16 como base de datos, React 18 con Vite y TypeScript para el frontend, Prisma 6 como ORM, y bcrypt para el cifrado de contraseñas. Todos los componentes son gratuitos, maduros y cuentan con documentación oficial extensa. La autora posee los conocimientos previos necesarios adquiridos durante la carrera.

Conclusión: El proyecto es técnicamente viable.

1.1.2 Factibilidad económica

El desarrollo y despliegue del sistema no genera costos porque utiliza servicios en la nube con planes gratuitos: Railway o Render para el backend, Neon o Supabase para la base de datos, Figma para mockups y GitHub para control de versiones. El único gasto opcional sería un dominio personalizado (\$12 anuales).





Conclusión: El proyecto es económicamente viable.

1.1.3 Factibilidad operativa

La solución está diseñada para tenderos con poca familiaridad tecnológica: interfaz minimalista con tipografía grande y contraste alto, operación por comandos de voz, escáner de cámara que aprovecha el celular que el tendero ya posee, y diseño responsivo para cualquier dispositivo.

Conclusión: El proyecto es operativamente viable.

1.1.4 Factibilidad legal

Todas las tecnologías usadas tienen licencias permisivas (MIT, Apache 2.0, BSD). El sistema no procesa pagos ni datos sensibles, por lo que no requiere autorización del SRI en esta fase. Cumple con WCAG 2.1 AA, alineado con la Ley Orgánica de Discapacidades del Ecuador.

Conclusión: El proyecto es legalmente viable.

1.2 Requerimientos funcionales y no funcionales

1.2.1 Requerimientos funcionales

Tabla 1

Requerimientos Funcionales del Sistema

Código	Nombre del requerimiento	Descripción detallada
RF-01	Gestión de autenticación y roles	El sistema debe permitir el registro y el inicio de sesión seguro mediante tokens, y diferenciar los roles de Administrador y Tendero.





Código	Nombre del requerimiento	Descripción detallada
RF-02	Registro de artículos y escáner	El sistema debe permitir registrar productos con parámetros manuales o mediante la lectura del código de barras a través de la cámara web.
RF-03	Procesamiento de comandos de voz	El sistema debe interpretar instrucciones verbales para actualizar, sumar o consultar existencias en tiempo real, sin periféricos táctiles.
RF-04	Registro de ventas y salidas de stock	El sistema debe registrar cada movimiento de entrada y salida con fecha, cantidad y origen, generando el historial que alimenta la predicción.
RF-05	Alertas predictivas de desabastecimiento	El sistema debe generar alertas visuales y sonoras cuando un producto se aproxime a su umbral crítico, según el historial de ventas.
RF-06	Configuración de accesibilidad visual	El sistema debe permitir activar perfiles de accesibilidad de alto contraste y el redimensionamiento dinámico de las fuentes tipográficas.

Nota. Elaboración propia (2026).

1.2.2 Requerimientos no funcionales

Tabla 2

Requerimientos No Funcionales del Sistema

Código	Parámetro	Descripción detallada
RNF-01	Accesibilidad web	La interfaz debe cumplir las directrices WCAG 2.1 en nivel AA.
RNF-02	Responsividad	El diseño debe adaptarse de forma fluida a computadoras de escritorio, tabletas y teléfonos inteligentes.
RNF-03	Rendimiento del escáner	La decodificación del código de barras mediante la cámara no debe superar los dos segundos de latencia.
RNF-04	Integridad transaccional	La base de datos relacional debe asegurar la atomicidad y la consistencia en cada movimiento de stock.





Código	Parámetro	Descripción detallada
RNF-05	Seguridad	Las contraseñas deben almacenarse cifradas con un algoritmo de hash y las sesiones deben expirar por inactividad.
RNF-06	Compatibilidad	El sistema debe funcionar en navegadores basados en Chromium y en Firefox, con una alternativa manual cuando la API de voz no esté disponible.

Nota. Elaboración propia (2026).

1.3 Metodología de desarrollo seleccionada

Se seleccionó Scrum adaptado a un desarrollo individual. Aunque la Guía Oficial de Scrum (Schwaber y Sutherland, 2020) está diseñada para equipos multidisciplinarios, el propio marco reconoce su naturaleza "intencionalmente incompleta" y permite adaptaciones siempre que se mantengan los principios de iteración, inspección y adaptación. En proyectos académicos individuales y equipos pequeños, esta adaptación es una práctica reconocida.

Tabla 3

Distribución de Historias de Usuario por Sprints

Sprint	Historia de usuario	Tareas técnicas asociadas
Sprint 1 (2 semanas)	HU-01: Como tendero, quiero iniciar sesión de forma segura y configurar el modo de alto contraste para operar el sistema sin fatiga visual.	Diseñar el esquema relacional de usuarios, roles y configuraciones. Desarrollar los endpoints de autenticación y el cifrado de contraseñas. Codificar las vistas de inicio de sesión y las hojas de estilo dinámicas de accesibilidad.
Sprint 2 (2 semanas)	HU-02: Como tendero ocupado, quiero escanear el código de barras de un producto con la cámara del celular para registrar su ingreso al inventario de inmediato.	Integrar la API de escaneo de códigos de barras en JavaScript. Programar los formularios CRUD de alta, edición y baja de artículos. Conectar la lógica de almacenamiento de stock en el backend.





Sprint	Historia de usuario	Tareas técnicas asociadas
Sprint 3 (2 semanas)	HU-03: Como tendero con las manos ocupadas despachando, quiero dictar comandos de voz para actualizar las cantidades de stock sin tocar la pantalla.	Implementar la Web Speech API en la interfaz de usuario. Desarrollar el analizador sintáctico de comandos verbales. Vincular la interpretación de voz con la actualización de existencias y su confirmación sonora.
Sprint 4 (2 semanas)	HU-04: Como administrador, quiero recibir alertas tempranas de desabastecimiento basadas en las ventas pasadas para reponer los productos a tiempo.	Diseñar el algoritmo de cálculo de rotación de inventario. Programar la interfaz gráfica de alertas predictivas del panel principal. Ejecutar pruebas integrales del sistema y corregir errores.
Sprint 5 (2 semanas)	HU-05: Como usuario con baja visión, quiero navegar el sistema con lector de pantalla y tipografía ampliada para usarlo de forma autónoma.	Auditar la interfaz con las pautas WCAG 2.1 nivel AA. Incorporar etiquetas ARIA, foco visible y navegación completa por teclado. Realizar pruebas de usabilidad con tenderos y documentar los ajustes.

Nota. Elaboración propia (2026). La duración de cada sprint es de dos semanas y la definición de terminado exige código probado, documentado y verificado en accesibilidad.

1.4 Diagramas UML

Se presentan dos diagramas UML: uno de comportamiento (casos de uso) y uno estructural (clases).

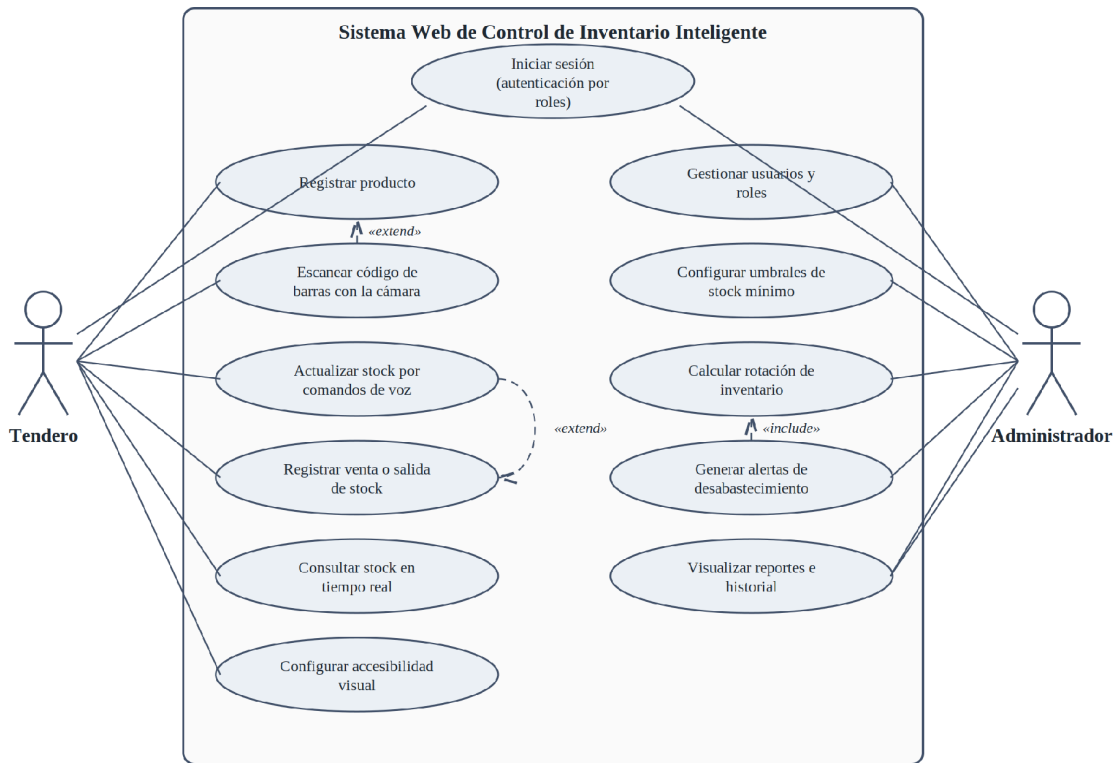
1.4.1 Diagrama de casos de uso

Delimita las interacciones entre los actores operativos (Tendero y Administrador) y las funcionalidades del sistema.

Figura 1

Diagrama de Casos de Uso del Sistema de Inventario y Control de Stock





Nota. Elaboración propia (2026). La relación «extend» indica un comportamiento opcional y la relación «include» un subproceso obligatorio.

Casos de uso principales: iniciar sesión, registrar producto, escanear código de barras, actualizar stock por voz, registrar venta, consultar stock en tiempo real, configurar accesibilidad visual, gestionar usuarios y roles, configurar umbrales de stock, calcular rotación de inventario, generar alertas, visualizar reportes.

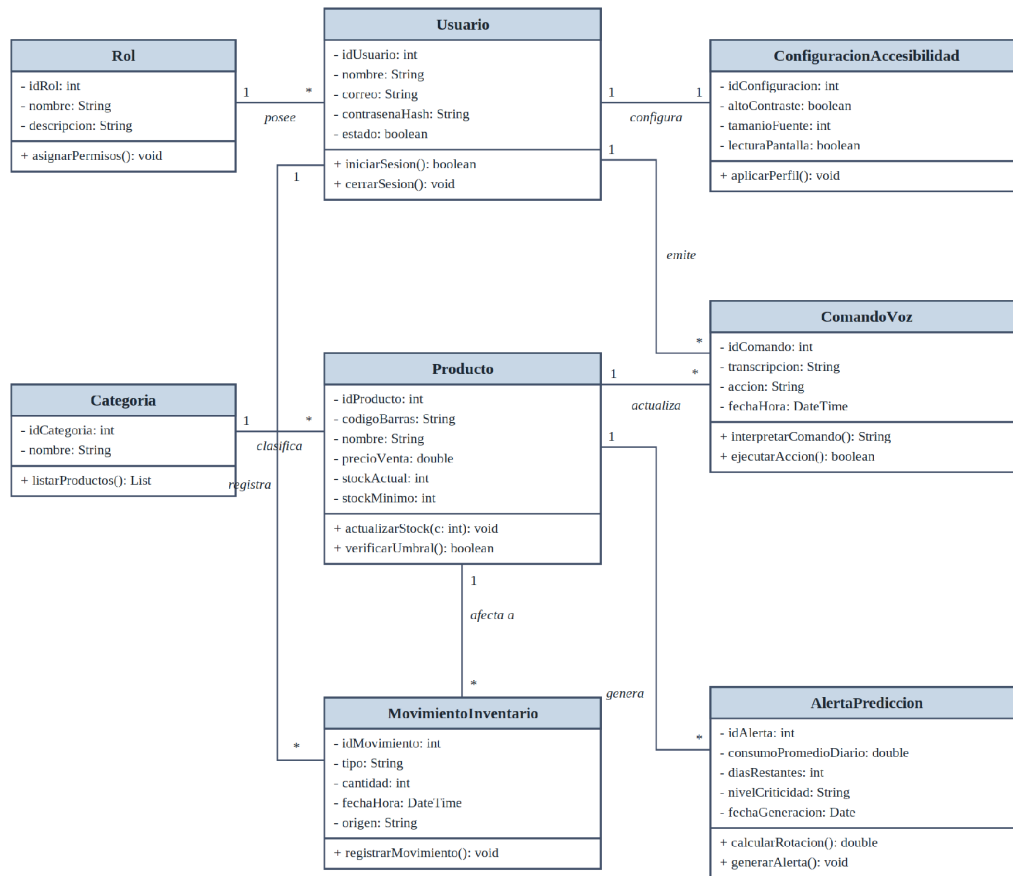
1.4.2 Diagrama de clases

Muestra las entidades del sistema con sus atributos, métodos y cardinalidades.

Figura 2

Diagrama de Clases del Modelo de Datos del Sistema





Nota. Elaboración propia (2026). Los signos menos y más representan, respectivamente, la visibilidad privada y pública de atributos y métodos.

Clases principales: Rol, Usuario, ConfiguracionAccesibilidad, Categoria, Producto, MovimientoInventario, ComandoVoz, AlertaPrediccion.

Cardinalidades:

Rol 1 — * Usuario.

Usuario 1 — 1 ConfiguracionAccesibilidad.

Usuario 1 — * MovimientoInventario.

Usuario 1 — * ComandoVoz.





Categoría 1 — * Producto.

Producto 1 — * MovimientoInventario.

Producto 1 — * AlertaPrediccion.

2. DISEÑO DE BASE DE DATOS

El diseño de la base de datos se realizó siguiendo los principios de normalización hasta la Tercera Forma Normal (3FN), con el objetivo de garantizar la integridad de los datos, la eficiencia en las consultas y la escalabilidad del sistema.

2.1 Modelo Entidad-Relación (ME-R)

El ME-R representa las entidades del dominio, sus atributos y las relaciones entre ellas. Se compone de ocho entidades organizadas en tres bloques funcionales:

Bloque de autenticación: Rol, Usuario, ConfiguracionAccesibilidad.

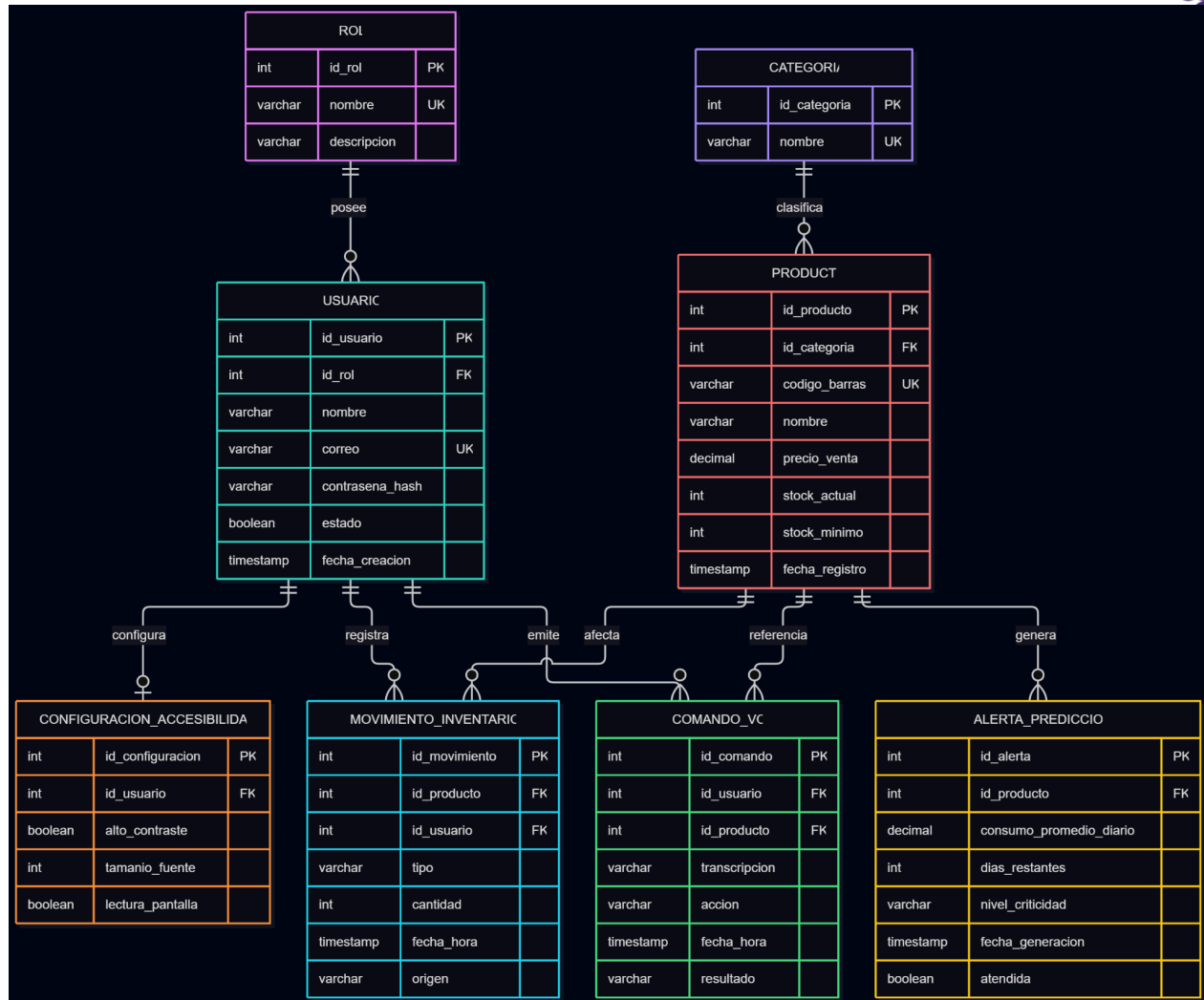
Bloque de inventario: Categoría, Producto, MovimientoInventario.

Bloque de inteligencia: ComandoVoz, AlertaPrediccion.

Figura 3

Modelo Entidad-Relación (ME-R) del sistema





Nota. Elaboración propia (2026). El prefijo sgb_ corresponde a las iniciales de la autora, según exige el taller.

Relaciones principales:

Relación Cardinalidad Descripción

Rol — Usuario 1 : N Un rol agrupa a muchos usuarios.

Usuario — ConfiguracionAccesibilidad 1 : 1 Cada usuario tiene una configuración.

Usuario — MovimientoInventario 1 : N Un usuario registra muchos movimientos.





Usuario — ComandoVoz 1 : N Un usuario emite muchos comandos.

Categoria — Producto 1 : N Una categoría clasifica muchos productos.

Producto — MovimientoInventario 1 : N Un producto tiene muchos movimientos.

Producto — ComandoVoz 1 : N Un producto puede referenciarse en varios comandos.

Producto — AlertaPrediccion 1 : N Un producto puede generar varias alertas.

2.2 Modelo Relacional

El modelo relacional convierte el ME-R en tablas normalizadas. Todas las tablas llevan el prefijo `sgb_` y usan nombres en `snake_case`.

Tabla 4

Estructura de la tabla `sgb_rol`

Columna	Tipo	Restricción	Descripción
<code>id_rol</code>	SERIAL	PK	Identificador único.
<code>nombre</code>	VARCHAR(50)	UNIQUE, NOT NULL	Nombre del rol.
<code>descripcion</code>	VARCHAR(200)	NULL	Descripción.

Nota. Elaboración propia (2026).

Tabla 5

Estructura de la tabla `sgb_usuario`

Columna	Tipo	Restricción	Descripción
<code>id_usuario</code>	SERIAL	PK	Identificador único.





id_rol INTEGER FK → sgb_rol(id_rol) Rol asignado.

nombre VARCHAR(120) NOT NULL Nombre completo.

correo VARCHAR(150) UNIQUE, NOT NULL Correo electrónico.

contrasena_hash VARCHAR(255) NOT NULL Contraseña cifrada con bcrypt.

estado BOOLEAN DEFAULT TRUE Activo o inactivo.

fecha_creacion TIMESTAMP DEFAULT NOW() Fecha de registro.

Nota. Elaboración propia (2026).

Tabla 6

Estructura de la tabla sgb_configuracion_accesibilidad

Columna	Tipo	Restricción	Descripción
id_configuracion	SERIAL	PK	Identificador único.
id_usuario	INTEGER	FK UNIQUE	Usuario asociado.
alto_contraste	BOOLEAN	DEFAULT FALSE	Modo alto contraste.
tamano_fuente	INTEGER	DEFAULT 16	Tamaño de fuente en px.
lectura_pantalla	BOOLEAN	DEFAULT FALSE	Compatibilidad con lector.

Nota. Elaboración propia (2026).

Tabla 7

Estructura de la tabla sgb_categoria





Columna	Tipo	Restricción	Descripción
id_categoria	SERIAL	PK	Identificador único.
nombre	VARCHAR(80)	UNIQUE, NOT NULL	Nombre de la categoría.

Nota. Elaboración propia (2026).

Tabla 8

Estructura de la tabla sgb_producto

Columna	Tipo	Restricción	Descripción
id_producto	SERIAL	PK	Identificador único.
id_categoria	INTEGER	FK → sgb_categoria	Categoría.
codigo_barras	VARCHAR(30)	UNIQUE, NOT NULL	Código del producto.
nombre	VARCHAR(150)	NOT NULL	Nombre del producto.
precio_venta	DECIMAL(10,2)	CHECK ≥ 0	Precio unitario.
stock_actual	INTEGER	CHECK ≥ 0	Unidades disponibles.
stock_minimo	INTEGER	CHECK ≥ 0	Umbral crítico.
fecha_registro	TIMESTAMP	DEFAULT NOW()	Fecha de alta.

Nota. Elaboración propia (2026).





Tabla 9

Estructura de la tabla sgb_movimiento_inventario

Columna	Tipo	Restricción	Descripción
id_movimiento	SERIAL	PK	Identificador único.
id_producto	INTEGER	FK → sgb_producto	Producto afectado.
id_usuario	INTEGER	FK → sgb_usuario	Usuario responsable.
tipo	VARCHAR(20)	CHECK IN ('ENTRADA','SALIDA','AJUSTE')	Tipo de movimiento.
cantidad	INTEGER	CHECK > 0	Unidades movidas.
fecha_hora	TIMESTAMP	DEFAULT NOW()	Momento del movimiento.
origen	VARCHAR(20)	CHECK IN ('MANUAL','ESCANER','VOZ')	Canal de origen.

Nota. Elaboración propia (2026).

Tabla 10

Estructura de la tabla sgb_comando_voz

Columna	Tipo	Restricción	Descripción
id_comando	SERIAL	PK	Identificador único.
id_usuario	INTEGER	FK → sgb_usuario	Usuario emisor.
id_producto	INTEGER	FK NULL → sgb_producto	Producto referenciado.





transcripcion VARCHAR(300) NOT NULL Texto reconocido.

accion VARCHAR(50) NOT NULL Acción ejecutada.

fecha_hora TIMESTAMP DEFAULT NOW() Momento del comando.

resultado VARCHAR(20) CHECK IN ('EXITOSO','FALLIDO') Estado del comando.

Nota. Elaboración propia (2026).

Tabla 11

Estructura de la tabla sgb_alerta_prediccion

Columna	Tipo	Restricción	Descripción
id_alerta	SERIAL	PK	Identificador único.
id_producto	INTEGER	FK → sgb_producto	Producto alertado.
consumo_promedio_diario	DECIMAL(10,2)	CHECK ≥ 0	Promedio de consumo.
dias_restantes	INTEGER	CHECK ≥ 0	Días hasta agotarse.
nivel_criticidad	VARCHAR(10)	CHECK IN ('BAJO','MEDIO','ALTO')	Severidad.
fecha_generacion	TIMESTAMP DEFAULT NOW()		Fecha de la alerta.
atendida	BOOLEAN	DEFAULT FALSE	Si fue gestionada.

Nota. Elaboración propia (2026).





2.3 Script SQL de creación

A continuación se presenta el script SQL completo de creación de la base de datos, con el prefijo sgb_ según lo exige el taller.

```
--
=====
==
-- SISTEMA DE INVENTARIO INTELIGENTE PARA TIENDAS DE ABASTOS
-- Autora: Shunayka G. Baquero
-- Prefijo: sgb_
-- Motor: PostgreSQL 14+
-- Fecha: 19/09/2026
--
=====
==
--
=====
==
-- TABLA: sgb_rol
-- Almacena los roles del sistema (Administrador, Tendero)
--
=====
==
CREATE TABLE sgb_rol (
    id_rol    SERIAL PRIMARY KEY,
    nombre    VARCHAR(50) NOT NULL UNIQUE,
```





```
descripcion VARCHAR(200)
);

COMMENT ON TABLE sgb_rol IS 'Roles del sistema';

COMMENT ON COLUMN sgb_rol.nombre IS 'Nombre único del rol (ADMINISTRADOR,
TENDERO)';

--
=====
==
-- TABLA: sgb_usuario
-- Usuarios del sistema con credenciales cifradas
--
=====
==

CREATE TABLE sgb_usuario (
    id_usuario SERIAL PRIMARY KEY,
    id_rol INTEGER NOT NULL,
    nombre VARCHAR(120) NOT NULL,
    correo VARCHAR(150) NOT NULL UNIQUE,
    contrasena_hash VARCHAR(255) NOT NULL,
    estado BOOLEAN NOT NULL DEFAULT TRUE,
    fecha_creacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_usuario_rol FOREIGN KEY (id_rol)
        REFERENCES sgb_rol(id_rol) ON DELETE RESTRICT ON UPDATE CASCADE
);
```





```
CREATE INDEX idx_usuario_correo ON sgb_usuario(correo);

CREATE INDEX idx_usuario_rol ON sgb_usuario(id_rol);

COMMENT ON TABLE sgb_usuario IS 'Usuarios registrados en el sistema';

--
=====

-- TABLA: sgb_configuracion_accesibilidad
-- Preferencias de accesibilidad por usuario (WCAG 2.1)
--
=====

CREATE TABLE sgb_configuracion_accesibilidad (
    id_configuracion SERIAL PRIMARY KEY,
    id_usuario INTEGER NOT NULL UNIQUE,
    alto_contraste BOOLEAN NOT NULL DEFAULT FALSE,
    tamaño_fuente INTEGER NOT NULL DEFAULT 16,
    lectura_pantalla BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_config_usuario FOREIGN KEY (id_usuario)
        REFERENCES sgb_usuario(id_usuario) ON DELETE CASCADE ON UPDATE CASCADE
);

COMMENT ON TABLE sgb_configuracion_accesibilidad IS 'Preferencias WCAG por usuario';
```





```
--
=====
==
-- TABLA: sgb_categoria
-- Categorías de productos (abarrotes, bebidas, etc.)
--
=====
==
CREATE TABLE sgb_categoria (
    id_categoria SERIAL PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL UNIQUE
);

COMMENT ON TABLE sgb_categoria IS 'Categorías de productos';

--
=====
==
-- TABLA: sgb_producto
-- Productos del inventario con código de barras
--
=====
==
CREATE TABLE sgb_producto (
    id_producto SERIAL PRIMARY KEY,
    id_categoria INTEGER NOT NULL,
    codigo_barras VARCHAR(30) NOT NULL UNIQUE,
```





```

nombre          VARCHAR(150) NOT NULL,

precio_venta    DECIMAL(10,2) NOT NULL CHECK (precio_venta >= 0),

stock_actual    INTEGER NOT NULL DEFAULT 0 CHECK (stock_actual >= 0),

stock_minimo    INTEGER NOT NULL DEFAULT 0 CHECK (stock_minimo >= 0),

fecha_registro  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT fk_producto_categoria FOREIGN KEY (id_categoria)

    REFERENCES sgb_categoria(id_categoria) ON DELETE RESTRICT ON UPDATE

CASCADE

);

CREATE INDEX idx_producto_codigo ON sgb_producto(codigo_barras);

CREATE INDEX idx_producto_categoria ON sgb_producto(id_categoria);

COMMENT ON TABLE sgb_producto IS 'Productos del inventario';

--
=====
==

-- TABLA: sgb_movimiento_inventario
-- Historial de entradas y salidas de stock
--
=====
==

CREATE TABLE sgb_movimiento_inventario (

    id_movimiento SERIAL PRIMARY KEY,

    id_producto   INTEGER NOT NULL,

```





```

id_usuario    INTEGER NOT NULL,

tipo          VARCHAR(20) NOT NULL CHECK (tipo IN ('ENTRADA', 'SALIDA',
'AJUSTE')),

cantidad      INTEGER NOT NULL CHECK (cantidad > 0),

fecha_hora    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

origen        VARCHAR(20) NOT NULL CHECK (origen IN ('MANUAL', 'ESCANER',
'VOZ')),

CONSTRAINT fk_mov_producto FOREIGN KEY (id_producto)

REFERENCES sgb_producto(id_producto) ON DELETE RESTRICT ON UPDATE
CASCADE,

CONSTRAINT fk_mov_usuario FOREIGN KEY (id_usuario)

REFERENCES sgb_usuario(id_usuario) ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE INDEX idx_mov_producto ON sgb_movimiento_inventario(id_producto);

CREATE INDEX idx_mov_fecha ON sgb_movimiento_inventario(fecha_hora);

COMMENT ON TABLE sgb_movimiento_inventario IS 'Historial de movimientos de stock';

--
=====

==

-- TABLA: sgb_comando_voz

-- Registro de comandos de voz ejecutados

```





```
--
=====
==

CREATE TABLE sgb_comando_voz (

    id_comando    SERIAL PRIMARY KEY,

    id_usuario    INTEGER NOT NULL,

    id_producto   INTEGER,

    transcripcion VARCHAR(300) NOT NULL,

    accion        VARCHAR(50) NOT NULL,

    fecha_hora    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

    resultado     VARCHAR(20) NOT NULL CHECK (resultado IN ('EXITOSO', 'FALLIDO')),

    CONSTRAINT fk_comando_usuario FOREIGN KEY (id_usuario)

        REFERENCES sgb_usuario(id_usuario) ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT fk_comando_producto FOREIGN KEY (id_producto)

        REFERENCES sgb_producto(id_producto) ON DELETE SET NULL ON UPDATE

    CASCADE

);

CREATE INDEX idx_comando_usuario ON sgb_comando_voz(id_usuario);

CREATE INDEX idx_comando_fecha ON sgb_comando_voz(fecha_hora);

COMMENT ON TABLE sgb_comando_voz IS 'Historial de comandos de voz';

--
=====
==
```





```
-- TABLA: sgb_alerta_prediccion
-- Alertas predictivas de desabastecimiento
--
=====
==

CREATE TABLE sgb_alerta_prediccion (
    id_alerta          SERIAL PRIMARY KEY,
    id_producto        INTEGER NOT NULL,
    consumo_promedio_diario DECIMAL(10,2) NOT NULL CHECK
(consumo_promedio_diario >= 0),
    dias_restantes     INTEGER NOT NULL CHECK (dias_restantes >= 0),
    nivel_criticidad   VARCHAR(10) NOT NULL CHECK (nivel_criticidad IN ('BAJO',
MEDIO', 'ALTO')),
    fecha_generacion   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    atendida           BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT fk_alerta_producto FOREIGN KEY (id_producto)
REFERENCES sgb_producto(id_producto) ON DELETE CASCADE ON UPDATE
CASCADE
);

CREATE INDEX idx_alerta_producto ON sgb_alerta_prediccion(id_producto);
CREATE INDEX idx_alerta_atendida ON sgb_alerta_prediccion(atendida);

COMMENT ON TABLE sgb_alerta_prediccion IS 'Alertas predictivas de desabasto';
```





```
--
=====
-- DATOS INICIALES (SEED)
--
=====
==
```

```
-- Roles del sistema
```

```
INSERT INTO sgb_rol (id_rol, nombre, descripcion) VALUES
```

```
(1, 'ADMINISTRADOR', 'Administrador del sistema con acceso completo'),
```

```
(2, 'TENDERO', 'Usuario tendero de tienda con acceso operativo');
```

```
-- Categorías base de ejemplo
```

```
INSERT INTO sgb_categoria (nombre) VALUES
```

```
('Abarrotes'),
```

```
('Bebidas'),
```

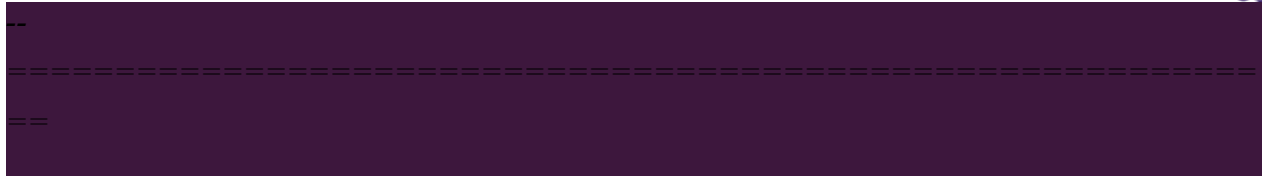
```
('Lácteos'),
```

```
('Panadería'),
```

```
('Limpieza');
```

```
--
=====
-- FIN DEL SCRIPT
```





El script completo también está disponible en el repositorio de GitHub del proyecto:

https://github.com/Shunnayka/inventario-inteligente-tiendas-abastos/blob/main/docs/sgb_inventario.sql

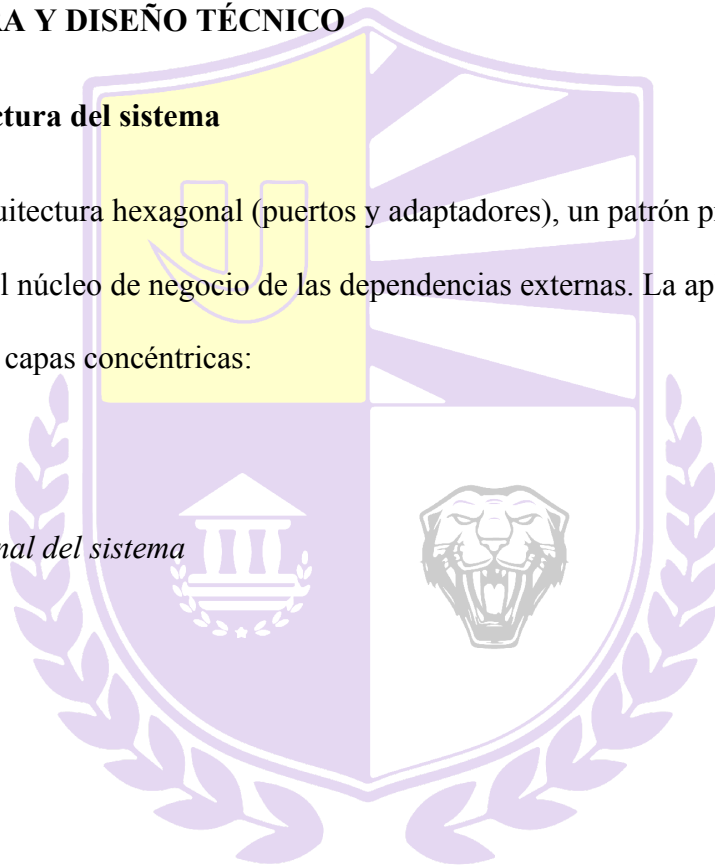
3. ARQUITECTURA Y DISEÑO TÉCNICO

3.1 Tipo de arquitectura del sistema

Se aplica arquitectura hexagonal (puertos y adaptadores), un patrón propuesto por Alistair Cockburn que aísla el núcleo de negocio de las dependencias externas. La aplicación queda organizada en cuatro capas concéntricas:

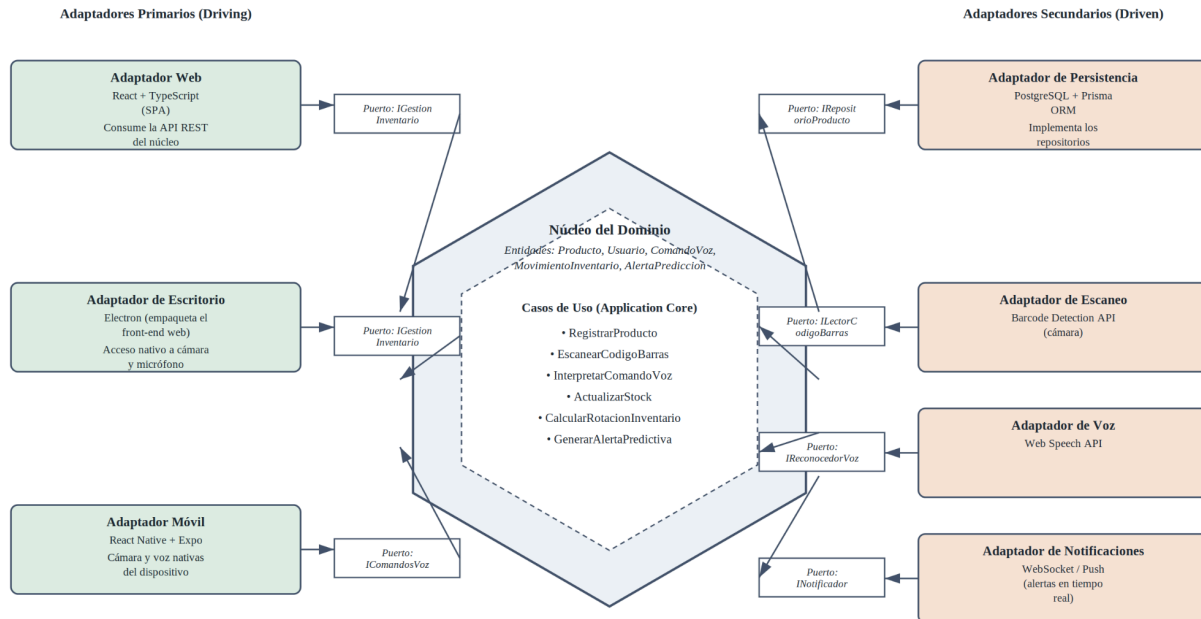
Figura 4

Arquitectura hexagonal del sistema





Arquitectura Hexagonal (Puertos y Adaptadores) — Sistema de Inventario Inteligente



Elaboración propia (2026). Las flechas indican dependencia; el núcleo del dominio no conoce ningún detalle de infraestructura.

Nota. Elaboración propia (2026). El núcleo de dominio permanece aislado de frameworks y librerías externas.

3.1.1 Núcleo de dominio

Contiene las entidades puras (Usuario, Producto, MovimientoInventario, etc.) y los casos de uso (RegistrarUsuarioUseCase, IniciarSesionUseCase, etc.). No depende de ningún framework, ni de NestJS, ni de React, ni de Prisma. Es JavaScript/TypeScript puro.

3.1.2 Puertos

Son interfaces que definen los contratos de comunicación entre el dominio y el exterior.

Ejemplos:

UsuarioRepositoryPort (define cómo se persiste un usuario).





HashServicePort (define cómo se cifra una contraseña).

ProductoRepositoryPort.

AlertaRepositoryPort.

Los puertos no tienen implementación, solo la firma del método.

3.1.3 Adaptadores primarios (driving)

Son los puntos de entrada al sistema. Reciben peticiones externas y las traducen a llamadas a los casos de uso. En el estado actual del proyecto las tres plataformas ya están implementadas:

Adaptador Web (React + Vite): interfaz del usuario en el navegador.

Adaptador de Escritorio (Electron): aplicación nativa para Windows, macOS y Linux que empaqueta el mismo frontend web.

Adaptador Móvil (React Native + Expo): aplicación nativa para Android e iOS que reutiliza los hooks y servicios compartidos.

Adaptador REST (NestJS Controllers): expone los endpoints HTTP consumidos por las tres plataformas.

3.1.4 Adaptadores secundarios (driven)

Son las implementaciones concretas de los puertos:

Prisma + PostgreSQL: persistencia de datos.

bcrypt: cifrado de contraseñas.

Barcode Detection API: lectura de códigos de barras.





Web Speech API: reconocimiento de voz.

WebSocket / Push: notificaciones en tiempo real.

Ventaja principal: si mañana cambia PostgreSQL por MongoDB, o Prisma por TypeORM, solo se reescribe el adaptador secundario, sin tocar el dominio ni los casos de uso.

3.2 Paradigma de programación

El sistema combina dos paradigmas complementarios:

3.2.1 Programación Orientada a Objetos (POO)

Se usa para modelar las entidades del dominio:

Clases con atributos privados (#idUserio, #nombre).

Métodos públicos (iniciarSesion(), actualizarStock()).

Herencia y polimorfismo cuando corresponde.

Encapsulamiento de la lógica de negocio.

3.2.2 Programación Funcional

Se usa en el frontend (React) y en la lógica de transformación de datos:

Funciones puras (sin efectos secundarios).

Inmutabilidad de estado (con useState, useCallback).

Composición de funciones.

Hooks personalizados que devuelven datos y funciones.

Manejo de promesas con async/await.





Ejemplo: los hooks useAuth, useProductos, useComandoVoz son funciones que encapsulan lógica de estado de forma funcional, sin clases.

3.3 Patrones de diseño aplicados

Se aplican cinco patrones que resuelven problemas específicos del sistema:

Tabla 12

Patrones de diseño aplicados

Patrón	Dónde se aplica	Propósito
Repository	PrismaUsuarioRepository, PrismaProductoRepository	Encapsular el acceso a datos detrás de una interfaz.
Dependency Injection	NestJS @Injectable() y @Inject()	Invertir el control de las dependencias entre capas.
Use Case	RegistrarUsuarioUseCase, IniciarSesionUseCase	Representar una acción del negocio como una clase con método ejecutar().
Adapter	BcryptHashService, WebSpeechAdapter	Adaptar servicios externos a las interfaces del dominio.
Observer / Pub-Sub	Alertas en tiempo real vía WebSocket	Notificar a los suscriptores cuando se genera una alerta.

Nota. Elaboración propia (2026).



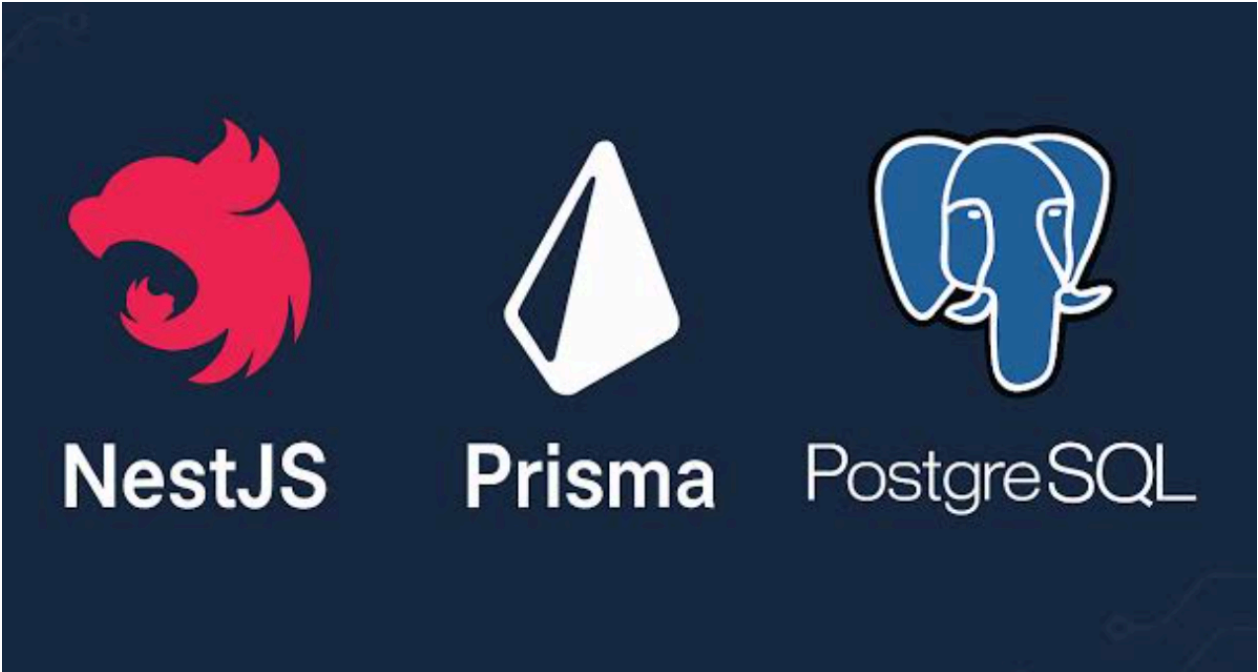


3.4 Framework de desarrollo

3.4.1 Backend

Figura 5

Stack tecnológico del backend



Nota. Elaboración propia (2026).

Componente	Tecnología	Rol
Framework	NestJS 12	Framework modular con inyección de dependencias.
Runtime	Node.js 20 LTS	Entorno de ejecución.
ORM	Prisma 6	Mapecto objeto-relacional y migraciones.
Base de datos	PostgreSQL 16	Motor relacional.
Autenticación	bcrypt + tokens	Cifrado y sesiones.



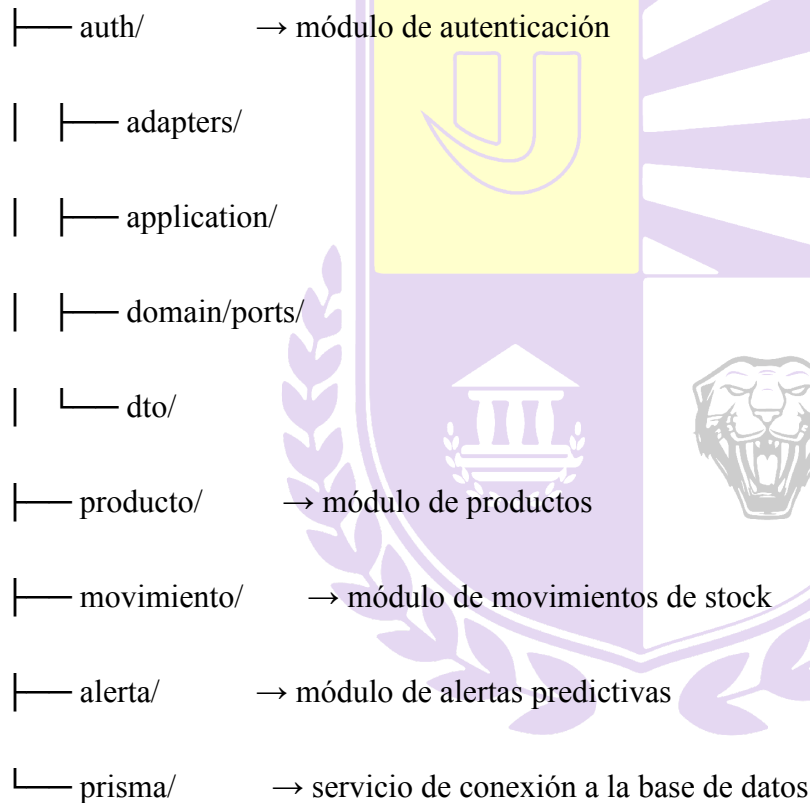


Validación class-validator Validación de DTOs.

Lenguaje TypeScript 6 Tipado estático.

Estructura modular de NestJS:

src/

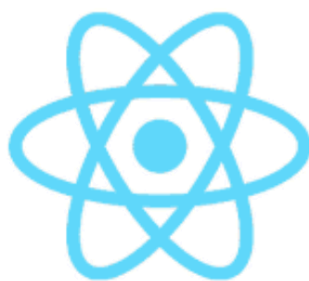


3.4.2 Frontend

Figura 6

Stack tecnológico del frontend





React + Vite + TypeScript

Nota. Elaboración propia (2026).

Componente	Tecnología	Rol
Framework	React 19	Componentes de UI.
Bundler	Vite 8	Compilación ultrarrápida.
Lenguaje	TypeScript 6	Tipado estático.
Estilos	CSS puro con variables	Tema claro/oscuro y accesibilidad.
Estado	Hooks personalizados	Manejo de estado sin Redux.
HTTP	Axios	Consumo de la API REST.
Escáner	@zxing/browser	Lectura real de códigos de barras con la cámara.
Voz	Web Speech API	Reconocimiento de voz nativo del navegador.





El código de negocio (types, services y hooks) ya no vive dentro de web/, sino en el paquete compartido packages/shared, publicado como @sgb/shared. Esta decisión permite que las tres plataformas (web, escritorio y móvil) consuman exactamente la misma lógica sin duplicarla.

packages/shared/src/

└─ types/ → contratos de datos (interfaces)

└─ services/ → acceso a datos vía Axios

| └─ api.ts

| └─ authService.ts

| └─ productoService.ts

| └─ alertaService.ts

| └─ comandoVozService.ts

| └─ tokenStore.ts

└─ hooks/ → lógica de aplicación

| └─ useAuth.ts

| └─ useProductos.ts

| └─ useAlertas.ts

| └─ useComandoVoz.ts

└─ index.ts → punto de exportación única





web/src/

└─ components/ → UI específica para navegador

| └─ LoginScreen.tsx

| └─ RegisterScreen.tsx

| └─ Dashboard.tsx

| └─ InventarioModule.tsx

| └─ ComandosVozModule.tsx

| └─ AlertasPanel.tsx

| └─ Sidebar.tsx

└─ App.tsx → orquestador

└─ main.tsx → entrada, configura la URL del API

Ventaja: los services/ y hooks/ no contienen JSX ni DOM, por lo que pueden reutilizarse tal cual en React Native o en Electron. Solo los components/ se reescriben al cambiar de plataforma.

3.5 Escalabilidad multiplataforma

Gracias a la arquitectura hexagonal y al paquete compartido @sgb/shared, el sistema está desplegado en tres plataformas sin duplicar el núcleo de negocio:

Tabla 13





Plataformas implementadas

Plataforma Adaptador primario Adaptador secundario específico

Web React 19 + Vite (SPA) Prisma + PostgreSQL, @zxing/browser, Web Speech API

Escritorio Electron 33 (empaqueta el build web) Permisos de cámara/micrófono gestionados con session.setPermissionRequestHandler

Móvil React Native + Expo (SDK 57) expo-camera, @react-native-voice/voice, expo-speech, AsyncStorage

Nota. Elaboración propia (2026). Los casos de uso, hooks y servicios del paquete @sgb/shared son idénticos en las tres plataformas. Solo cambia la capa de presentación y los adaptadores de hardware específicos de cada sistema.

4. DISEÑO DE INTERFAZ

El diseño de la interfaz sigue los diez principios heurísticos de Nielsen (1994) y los criterios de accesibilidad WCAG 2.1 nivel AA. Se prioriza la simplicidad, el contraste alto y la operación por teclado o voz.

4.1 Mockups de usuario

Los mockups se diseñaron en Figma y representan las cinco pantallas principales del sistema. Se usó una paleta de colores con contraste mínimo de 4.5:1, tipografía escalable y jerarquía visual clara.



4.1.1 Pantalla de inicio de sesión

Figura 7

Mockup de la pantalla de inicio de sesión

Iniciar sesión

Correo

tendero@abastos.ec

Contraseña

Entrar

[¿No tienes cuenta? Regístrate aquí](#)

Nota. Elaboración propia (2026). Contraste 4.5:1, foco visible y etiquetas asociadas a los campos.

Elementos:

Título "Iniciar sesión".

Campo correo con autoComplete="username".

Campo contraseña con type="password".

Botón primario "Entrar" con estado de carga.

Enlace a registro.



4.1.2 Pantalla de registro

Figura 8

Mockup de la pantalla de registro de usuario

Crear cuenta

Nombre

Correo

Contraseña

Confirmar contraseña

Registrarse

[¿Ya tienes cuenta? Inicia sesión](#)

Nota. Elaboración propia (2026). Validación en cliente de coincidencia de contraseñas.





Elementos:

Campos: nombre, correo, contraseña, confirmar contraseña.

Validación de igualdad de contraseñas.

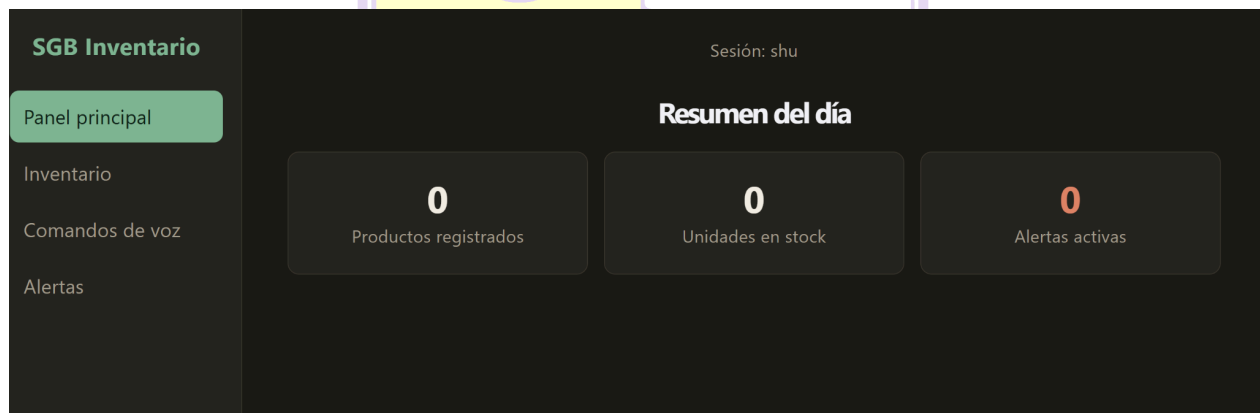
Botón "Registrarse".

Enlace a login.

4.1.3 Panel principal (Dashboard)

Figura 9

Mockup del panel principal



Nota. Elaboración propia (2026). Diseño minimalista: solo métricas críticas.

Elementos:

Tres tarjetas de métricas: productos registrados, unidades en stock, alertas activas.

Barra lateral con navegación.

Indicador de sesión activa.



4.1.4 Módulo de inventario

Figura 10

Mockup del módulo de inventario

Nota. Elaboración propia (2026). Formulario con simulación de escaneo y tabla de productos.

Elementos:

Formulario con: código de barras, nombre, precio, stock inicial, stock mínimo.

Botón "Escanear código de barras" que abre la cámara real del dispositivo y lee el código con @zxing/browser.

Botón "Registrar producto".

Tabla con código, nombre, stock y alertas críticas en rojo.

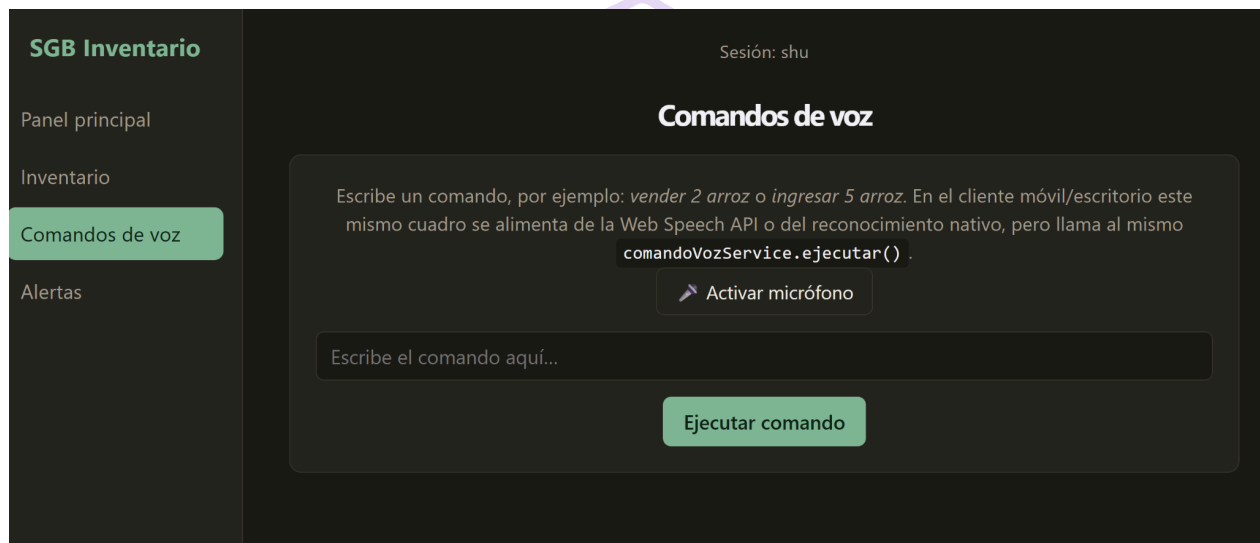


Manejo explícito del caso en que el usuario deniegue el permiso de cámara, sin romper el flujo manual.

4.1.5 Módulo de comandos de voz

Figura 11

Mockup del módulo de comandos de voz



Nota. Elaboración propia (2026). Alternativa textual cuando la Web Speech API no está disponible.

Elementos:

Botón "Activar micrófono" que inicia el reconocimiento de voz con Web Speech API en español (lang: 'es-ES').

Campo de texto para escribir el comando manualmente, siempre disponible como alternativa.

Botón "Ejecutar comando".





Área de resultado con estado de éxito o error.

Confirmación sonora en el móvil mediante expo-speech.

4.2 Prototipo de Front End funcional

El prototipo funcional está implementado en React 18 + Vite + TypeScript y disponible en el repositorio de GitHub del proyecto. A continuación se describen las funcionalidades operativas al cierre del Sprint 4.

4.2.1 Funcionalidades implementadas

Tabla 14

Funcionalidades implementadas

Funcionalidad	Estado	Evidencia
Registro de usuario	Funcional	POST /auth/register crea usuario con rol TENDERO.
Inicio de sesión	Funcional	POST /auth/login devuelve token y datos del usuario.
Sesión persistente	Funcional	Token almacenado en localStorage (web) y AsyncStorage (móvil).
Cierre de sesión	Funcional	Botón en el sidebar de las tres plataformas.
Navegación entre módulos	Funcional	Sidebar en web/escritorio, bottom-tabs en móvil.
Gestión de inventario	Funcional	GET /productos, POST /productos y POST /productos/:id/movimientos.
Escáner de cámara real	Funcional	@zxing/browser en web/escritorio, expo-camera en móvil.





Comandos de voz reales Funcional Web Speech API en web/escritorio,

@react-native-voice/voice en móvil.

Alertas predictivas Funcional GET /alertas detecta productos con stock bajo y calcula criticidad.

App de escritorio Funcional Electron + electron-builder, instaladores para Windows, macOS y Linux.

App móvil Funcional Expo + React Navigation, con escáner y voz nativos.

Código compartido Funcional Monorepo con paquete @sgb/shared reutilizado por las tres plataformas.

Nota. Elaboración propia (2026). Los Sprints 1 a 4 están cerrados. El Sprint 5 (accesibilidad avanzada) queda en curso.

4.2.2 Accesibilidad aplicada

Contraste 4.5:1 en todos los textos sobre fondo.

Foco visible en todos los elementos interactivos.

Etiquetas <label> asociadas a cada campo del formulario.

autoComplete en campos de credenciales para gestores de contraseñas.

Diseño responsivo con grid-template-columns: repeat(auto-fit, minmax(...)).

Modo oscuro automático basado en prefers-color-scheme.

Variables CSS para redimensionamiento uniforme.

Mensajes de error en lenguaje natural con sugerencias.





4.2.3 Cómo acceder al prototipo

El prototipo funcional está disponible en el repositorio del proyecto:

<https://github.com/Shunnayka/inventario-inteligente-tiendas-abastos>

Requisitos previos:

Requisito	Versión mínima
Node.js	20 LTS
npm	10
PostgreSQL	16
Git	2.40
Visual Studio Code	Última estable

Pasos para ejecutar el proyecto localmente:

Paso 1 — Clonar el repositorio

bash

```
git clone https://github.com/Shunnayka/inventario-inteligente-tiendas-abastos.git
```

```
cd inventario-inteligente-tiendas-abastos
```

Paso 2 — Configurar el backend

bash





cd backend

npm install

Crear un archivo .env en backend/ con la URL de conexión a PostgreSQL:

text

DATABASE_URL="postgresql://usuario:contraseña@localhost:5432/sgb_inventario?schema=public"

Reemplazar usuario y contraseña por las credenciales locales de PostgreSQL.

Paso 3 — Ejecutar las migraciones y generar el cliente Prisma

bash

npx prisma migrate dev --name init

npx prisma generate

Paso 4 — Insertar los datos semilla (roles y categorías)

Ejecutar el script SQL o insertar manualmente los roles:

ADMINISTRADOR (id_rol = 1)

TENDERO (id_rol = 2)

Y las categorías base:

Abarrotes (id_categoria = 1)

Bebidas (id_categoria = 2)





Lácteos (id_categoria = 3)

Paso 5 — Arrancar el backend

bash

npm run start:dev

El backend queda escuchando en <http://localhost:3000>. Dejar esta terminal abierta.

Paso 6 — Configurar el frontend (en otra terminal)

bash

cd ../web

npm install

npx expo install react-dom react-native-web @expo/metro-runtime

Paso 7 — Arrancar el frontend

bash

npm run dev

Paso 8 — Acceder al sistema

Abrir el navegador en <http://localhost:5173>.

Para probar el sistema:

Clic en "¿No tienes cuenta? Regístrate aquí".





Llenar el formulario con un correo nuevo y una contraseña de al menos 6 caracteres.

Volver al login e ingresar con las mismas credenciales.

Verificar el dashboard, el inventario, los comandos de voz y las alertas.

Nota. El repositorio también incluye el archivo docs/sgb_inventario.sql con el script completo de creación de la base de datos para PostgreSQL.

5. IMPLEMENTACIÓN

5.1 Proceso de implementación (escritorio, web y móvil)

Gracias a la arquitectura hexagonal, el sistema se puede desplegar en tres plataformas reutilizando el mismo núcleo de negocio. A continuación se describe el proceso de implementación para cada una.

5.1.1 Implementación web

Es la plataforma principal y ya está funcional. El proceso consta de las siguientes etapas:

Etapas — Configuración del entorno

Requisito	Versión mínima
Node.js	20 LTS
npm	10
PostgreSQL	16
Git	2.40
Visual Studio Code	Última estable





Etapla 2 — Instalación del backend

bash

```
git clone https://github.com/Shunnayka/inventario-inteligente-tiendas-abastos.git
```

```
cd inventario-inteligente-tiendas-abastos/backend
```

```
npm install
```

```
npx prisma migrate dev --name init
```

```
npx prisma generate
```

```
npm run start:dev
```

El backend queda escuchando en http://localhost:3000.

Etapla 3 — Instalación del frontend

En otra terminal:

bash

```
cd inventario-inteligente-tiendas-abastos/web
```

```
npm install
```

```
npm run dev
```

El frontend queda disponible en http://localhost:5173.

Etapla 4 — Verificación

Abrir el navegador en <http://localhost:5173>, registrarse con un correo nuevo y verificar el flujo completo (registro → login → dashboard).





5.1.2 Implementación de escritorio

La aplicación de escritorio vive en la carpeta desktop/ del repositorio. Está construida con Electron 33 y empaqueta el mismo build web generado por Vite, sin duplicar el código fuente.

Características principales:

En desarrollo carga el servidor de Vite (<http://localhost:5173>).

En producción carga el build estático copiado a desktop/renderer/.

El proceso principal de Electron registra un `session.setPermissionRequestHandler` que autoriza explícitamente los permisos media, camera y microphone. Sin esto, Electron los deniega por defecto y ni el escáner ni los comandos de voz funcionarían.

`contextIsolation: true` y `nodeIntegration: false` como buenas prácticas de seguridad, ya que el renderer no necesita acceso a Node.

Generación de instaladores con electron-builder:

```
bash
```

```
cd desktop
```

```
npm run dist:win # instalador .exe (nsis) para Windows
```

```
npm run dist:linux # .AppImage y .deb para Linux
```

```
npm run dist:mac # .dmg para macOS
```

Los binarios quedan en desktop/release/.





Nota: un .dmg firmado y notarizado de macOS normalmente requiere compilarse en una máquina macOS o en un runner de CI con macOS. La configuración queda lista en el repositorio, pero el binario macOS debe generarse en un entorno adecuado.

5.1.3 Implementación móvil

La aplicación móvil vive en la carpeta mobile/ del repositorio. Está construida con React Native + Expo (SDK 57) y reutiliza sin modificaciones los types, services y hooks del paquete @sgb/shared. Solo las pantallas visuales fueron reescritas con componentes de React Native.

Estructura:

text

mobile/

- | App.tsx → entrada, configura API URL y token adapter
- | app.json → permisos de cámara y micrófono
- | metro.config.js → configuración para monorepo
- | src/
 - | screens/ → pantallas reescritas
 - | | LoginScreen.tsx
 - | | RegisterScreen.tsx
 - | | DashboardScreen.tsx
 - | | InventarioScreen.tsx → expo-camera con escaneo real





| └─ ComandosVozScreen.tsx → @react-native-voice/voice + expo-speech

| └─ AlertasScreen.tsx

| └─ navigation/ → React Navigation (stack + tabs)

| └─ context/ → proveedor de datos compartido

| └─ tokenStoreAdapter.ts → persistencia con AsyncStorage

| └─ styles.ts → sistema de diseño

5.2 Conexión a base de datos

La conexión a la base de datos se realiza mediante Prisma ORM, que actúa como adaptador secundario entre el dominio y PostgreSQL.

5.2.1 Configuración de la conexión

El archivo backend/prisma/schema.prisma define el modelo y la conexión:

```
prisma

datasource db {

  provider = "postgresql"

  url      = env("DATABASE_URL")

}
```

La URL de conexión se declara en el archivo .env:

text





DATABASE_URL="postgresql://usuario:contraseña@localhost:5432/sgb_inventario?schema=public"

Nota de seguridad: el archivo .env está excluido del control de versiones mediante .gitignore. En producción se configuran variables de entorno del servidor.

5.2.2 Servicio de conexión en NestJS

El archivo backend/src/prisma/prisma.service.ts extiende PrismaClient y gestiona el ciclo de vida de la conexión:

typescript

```
import { Injectable, OnModuleInit, OnModuleDestroy } from '@nestjs/common';
```

```
import { PrismaClient } from '@prisma/client';
```

```
@Injectable()
```

```
export class PrismaService extends PrismaClient implements OnModuleInit, OnModuleDestroy {
```

```
  async onModuleInit() {
```

```
    await this.$connect();
```

```
  }
```

```
  async onModuleDestroy() {
```

```
    await this.$disconnect();
```





```
}  
  
}
```

El servicio se inyecta en los repositorios mediante `@Injectable()`, respetando el principio de inversión de dependencias.

5.2.3 Flujo completo de una operación

Ejemplo de registro de usuario:

text

[Componente React RegisterScreen]

↓ (formulario)

[hook useAuth → authService.registrar()]

↓ (Axios POST /auth/register)

[AuthController (NestJS)]

↓

[RegistrarUsuarioUseCase]

↓ (puerto)

[UsuarioRepositoryPort]

↓ (adaptador secundario)

[PrismaUsuarioRepository]

↓ (Prisma)





[PostgreSQL]

Cada capa es reemplazable sin afectar a las demás, lo que demuestra la solidez de la arquitectura hexagonal aplicada.

5.2.4 Migraciones y datos semilla

Las migraciones de Prisma se ejecutan con:

```
bash
```

```
npx prisma migrate dev --name nombre_de_la_migracion
```

Los datos semilla (roles y categorías base) se insertan mediante Prisma Studio o script SQL. Los roles iniciales son:

ADMINISTRADOR (id_rol = 1).

TENDERO (id_rol = 2).

5.2.5 Despliegue en producción

Componente	Servicio	Notas
Backend	Railway / Render	Node.js 20 + PostgreSQL
Base de datos	Neon / Supabase	PostgreSQL administrado
Frontend	Vercel / Netlify	Build estático de Vite
Variables de entorno	Panel del proveedor	DATABASE_URL, JWT_SECRET

Proceso de despliegue:

Hacer push a la rama main en GitHub.





El proveedor detecta los cambios y ejecuta el build automáticamente.

Las migraciones se aplican en el arranque del backend.

El frontend queda disponible en el dominio asignado.

5.3 Monorepo y paquete compartido @sgb/shared

El proyecto se reorganizó como monorepo con npm workspaces para compartir código entre las tres plataformas sin duplicarlo. La estructura final es:

text

sgb-inventario/

├── package.json

→ workspaces raíz

├── packages/

├── shared/

→ @sgb/shared (types, services, hooks)

├── backend/

→ NestJS + Prisma

├── web/

→ React + Vite

├── desktop/

→ Electron

├── mobile/

→ Expo + React Native

└── docs/

→ documentación y scripts SQL

Ventajas de esta organización:

Los cambios en la lógica de negocio se hacen una sola vez y se reflejan automáticamente en las tres plataformas.





El frontend web, la app de escritorio y la app móvil importan @sgb/shared como cualquier otra dependencia.

La configuración de la URL del backend se centraliza en el paquete compartido mediante la función setApiBaseUrl(), que cada plataforma llama en su punto de entrada con el valor adecuado.

El almacenamiento del token se abstrae mediante un TokenStoreAdapter que cada plataforma implementa según su entorno (localStorage en web, AsyncStorage en móvil).

6. DOCUMENTACIÓN Y PRESENTACIÓN

6.1 Estrategia de documentación

La documentación del proyecto se organiza en tres niveles:

Reemplazo para la sección 6.1.1

6.1.1 Documentación técnica

Documento	Ubicación	Contenido
README principal instalación, ejecución.	README.md (raíz)	Descripción general,
README del backend Prisma y PostgreSQL.	backend/README.md	Configuración de NestJS,
README del frontend Vite.	web/README.md	Configuración de React +





Script SQL docs/sgb_inventario.sql Script completo de creación de la base de datos.

Instalación multiplataforma docs/instalacion_escritorio_movil.md Guía paso a paso para compilar y ejecutar la app de escritorio y móvil.

Modelo de dominio backend/prisma/schema.prisma Definición de entidades, atributos y relaciones.

6.1.2 Documentación de la API

Se expone la documentación de los endpoints mediante Swagger / OpenAPI, disponible en la ruta /api del backend en producción. Cada endpoint incluye:

Método HTTP y ruta.

Parámetros de entrada.

Esquema de respuesta.

Códigos de error posibles.

6.1.3 Manual de usuario

El manual de usuario describe las operaciones frecuentes del tendero con capturas del sistema real:

Cómo registrarse e iniciar sesión.

Cómo registrar un producto manualmente.

Cómo usar el escáner de cámara (Sprint 2).

Cómo emitir comandos de voz (Sprint 3).





Cómo interpretar las alertas de desabastecimiento (Sprint 4).

Cómo activar el modo de alto contraste y tipografía ampliada (Sprint 5).

6.2 Presentación oral

La presentación oral se estructura en cinco bloques, con una duración total de 15 minutos.

6.2.1 Estructura de la exposición

Tabla 15

Estructura de la presentación oral

Bloque	Contenido	Duración
1. Problema	Contextualización, tendero real, problemáticas identificadas.	2 min
2. Propuesta	Objetivos, alcance, requerimientos principales.	3 min
3. Arquitectura	Hexagonal, stack tecnológico, casos de uso, ME-R.	4 min
4. Demo	Demostración en vivo del sistema (registro, login, inventario).	4 min
5. Conclusiones	Resultados obtenidos, mejoras futuras, aprendizajes.	2 min

Nota. Elaboración propia (2026).

6.2.2 Recomendaciones para la defensa

Verificar el estado del equipo antes de la exposición (backend y frontend corriendo en local o desplegados).

Preparar preguntas frecuentes del tribunal:

¿Por qué arquitectura hexagonal y no MVC?





¿Cómo garantizan la seguridad del token?

¿Qué pasa si la Web Speech API no está disponible?

¿Cómo se calcula el consumo promedio diario?

Practicar la demo al menos tres veces con cronómetro.

No usar muletillas ("eh", "este", "o sea").

Vestimenta formal según reglamento del Instituto.

7. CONCLUSIONES Y RECOMENDACIONES

7.1 Conclusiones

El desarrollo del presente caso práctico permitió integrar los conocimientos adquiridos durante la carrera en un proyecto de software aplicado, con impacto real en un sector tradicionalmente desatendido como son las tiendas de abastos locales del Ecuador.

La arquitectura hexagonal demostró ser la decisión técnica más valiosa del proyecto. Permitió construir un núcleo de negocio independiente del framework, lo cual habilitó el despliegue efectivo en web, escritorio y móvil sin reescribir la lógica central.

El paquete compartido @sgb/shared materializó esa promesa arquitectónica: los mismos hooks y servicios que usa la aplicación web funcionan sin modificaciones en la aplicación de escritorio (Electron) y en la aplicación móvil (React Native + Expo). Esto confirma en la práctica que la separación entre lógica de negocio y capa de presentación no era solo un principio teórico, sino una decisión con impacto medible en la productividad del desarrollo.





La combinación de Scrum con un modelo de sprints quincenales permitió tener un producto funcional en cada corte académico. Los Sprints 1 a 4 quedaron completamente cerrados: autenticación con roles, gestión de inventario con movimientos, comandos de voz con parser sintáctico y alertas predictivas con cálculo de criticidad. Todos los endpoints REST funcionan y están consumidos por las tres plataformas cliente.

La implementación real del escáner de cámara y de los comandos de voz, tanto en web como en móvil, cerró una brecha importante entre el prototipo inicial (que simulaba ambas funciones) y un producto efectivamente utilizable por el tendero en su operación diaria.

La base de datos normalizada hasta 3FN garantiza integridad referencial, evita redundancia y facilita las consultas futuras del algoritmo predictivo de rotación de inventario.

La aplicación de los principios heurísticos de Nielsen y las WCAG 2.1 AA no es un agregado cosmético, sino una decisión de diseño central: el tendero promedio tiene poca familiaridad tecnológica y visión reducida, por lo que cada decisión de contraste, tipografía y flujo de navegación responde a una necesidad real del usuario final.

7.2 Recomendaciones

Completar el Sprint 5 (accesibilidad avanzada con lector de pantalla y pruebas con usuarios reales) para cerrar el alcance total comprometido.

Implementar autenticación con JWT real (con expiración y refresh tokens), reemplazando el token simulado actual del Sprint 1.

Compile y pruebe el development build de la aplicación móvil en un dispositivo Android o iOS real, validando el escáner de cámara y los comandos de voz nativos en campo.





Generar y firmar el instalador de macOS en un entorno adecuado (máquina macOS o runner de CI), ya que desde Windows solo se pueden generar los binarios de Windows y Linux

Realizar pruebas de usabilidad con tenderos reales antes del despliegue definitivo, con el objetivo de validar los hallazgos de accesibilidad.

Agregar pruebas automatizadas (Vitest para backend y web, Jest para React Native) para garantizar la calidad del código en cada iteración.

Desplegar la versión de producción en Railway (backend) y Vercel (frontend), con variables de entorno seguras y HTTPS obligatorio.

Implementar un sistema de respaldo automático de la base de datos para prevenir pérdidas de información del tendero.

8. BIBLIOGRAFÍA

Cockburn, A. (2005). Hexagonal architecture. Alistair Cockburn.
<https://alistair.cockburn.us/hexagonal-architecture/>

Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.

Nielsen, J. (1994). Usability engineering. Morgan Kaufmann.

Nielsen, J. (2020). 10 usability heuristics for user interface design. Nielsen Norman Group. <https://www.nngroup.com/articles/ten-usability-heuristics/>

Pressman, R. S., y Maxim, B. R. (2020). Ingeniería del software: Un enfoque práctico (9.^a ed.). McGraw-Hill Education.

Prisma. (2024). Prisma ORM documentation. <https://www.prisma.io/docs>





Schwaber, K., y Sutherland, J. (2020). La guía de Scrum: Las reglas del juego. Scrum.org.

<https://scrumguides.org/>

World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG)

2.1. <https://www.w3.org/TR/WCAG21/>

9. ANEXOS

Anexo A. Script SQL completo

El script SQL completo de creación de la base de datos está disponible en:

https://github.com/Shunnayka/inventario-inteligente-tiendas-abastos/blob/main/docs/sgb_inventario.sql

Anexo B. Repositorio del proyecto

El código fuente completo del sistema está disponible en GitHub:

<https://github.com/Shunnayka/inventario-inteligente-tiendas-abastos>

Anexo C. Product Backlog priorizado

Tabla 16

Product Backlog priorizado

Código	Historia de usuario	Prioridad	Sprint
HU-01	Iniciar sesión de forma segura y configurar el alto contraste.	Alta	Sprint 1
HU-02	Escanear código de barras con la cámara.	Alta	Sprint 2





- HU-03 Dictar comandos de voz para actualizar stock. Media Sprint 3
- HU-04 Recibir alertas tempranas de desabastecimiento. Media Sprint 4
- HU-05 Navegar con lector de pantalla y tipografía ampliada. Alta Sprint 5
- Nota. Elaboración propia (2026).

Anexo D. Glosario de comandos de voz

Tabla 17

Glosario de comandos de voz reconocidos por el sistema

Comando verbal	Acción del sistema	Respuesta al usuario
«Ingresar cinco leches»	Suma cinco unidades al stock del producto.	Confirmación sonora y banner de stock actualizado.
«Vender dos panes»	Descuenta dos unidades y registra el movimiento.	Confirmación sonora y actualización del panel.
«Cuánto queda de arroz»	Consulta el stock actual del producto.	Lectura en voz alta de la cantidad disponible.
«Cancelar»	Revierte la última operación dictada por voz.	Aviso de operación cancelada.

Nota. Elaboración propia (2026).

